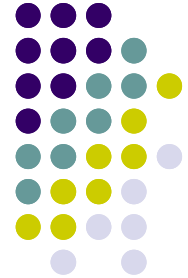


Architectural Process

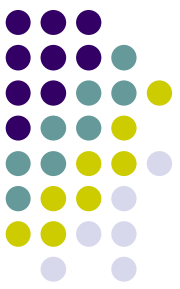


Architecture of Computer Games
SE 456

Ed Keenan

12 January 2022

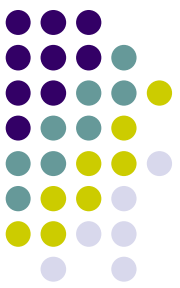
13.0.9.3.9 Mayan Long Count



Overview

- How to Design Software
- Unknown
- Architectural Designs
- Never Finished
- Books



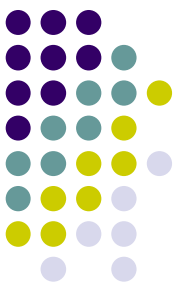


How to Design Software?

- I assumed everyone knows how to design
 - Is that a correct assumption?
- Where did you learn software design?
 - Which classes?
 - Did they really work?
- What's is your approach?
 - Please share
- What works for you?
- What doesn't work for you?

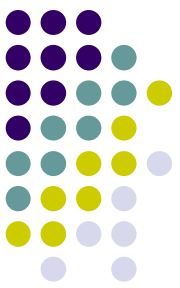


How do you Design the Unknown?



- Very interesting problem
 - Everyone has opinions
 - Easy / Hard?
 - Follow a process?
 - Get inspiration?
- It scares the shit out of me
 - Can I *really* solve it?
 - I'm being tested
 - I've been bluffing my way through software
 - Will I get caught?



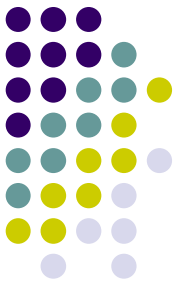


Here's my process

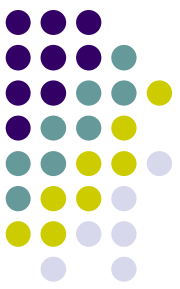
- It doesn't follow any **OFFICIAL** process or specific engineering patterns
 - I started like everyone else following software engineer process
- Process
 - Gather Requirements
 - Design
 - Implement
 - Test



Why doesn't that work?



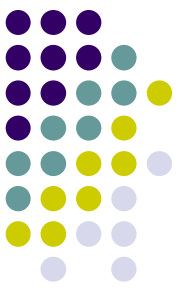
The Unknown



Architecture Design

1. Outline Features
2. Prototype to Learn
3. Indentify Use Cases
4. Tracer Bullets to target solution
5. Iterate



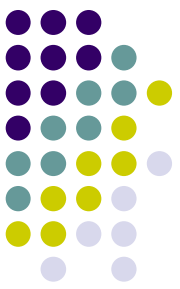


Outline Features

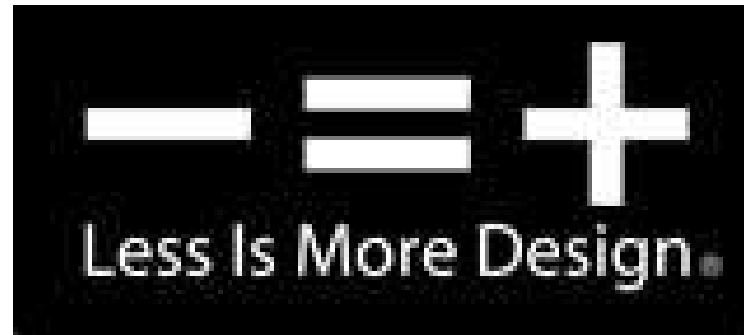
- Create a wish list of expected features
 - Don't think too futuristic
 - Be realistic
- Prioritize features
 - Must have
 - Nice to have



Outline Features Specs

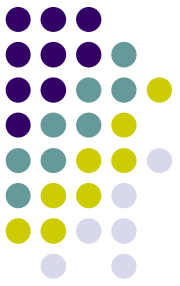


- Feature specification
 - Quick list of feature requests
 - Describe each feature enough to convey ideas on what you want
 - Don't be too specific

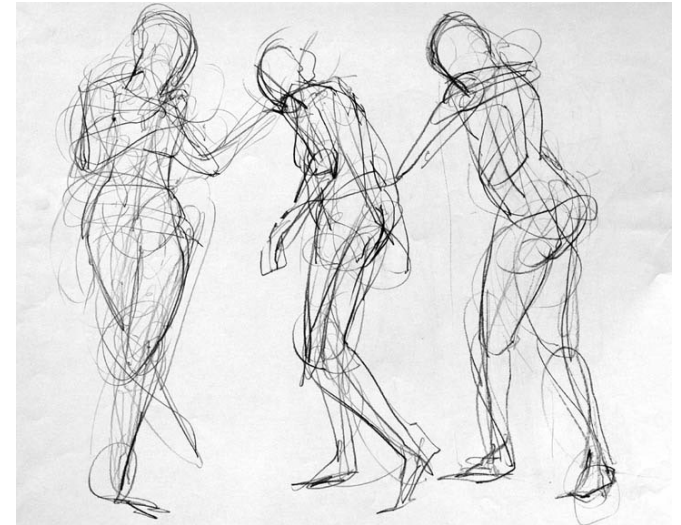


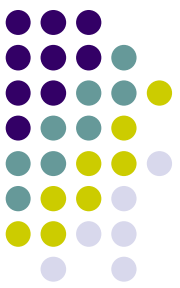
Outline Features

Your List



- List is for you
 - Quick pictures
 - Enough of details to remember
 - Not for others
- Sketch on how to use feature
 - Sample method call
- Whatever works for you
 - Short hand, doodles, formal write-ups...



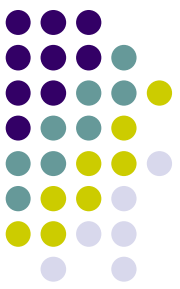


Prototype to Learn

- Learn and Gain Experience
 - Quickly
- Create many standalone programs
 - How do I ...
 - use the function?
 - create my data structure?
 - use x-feature of the language?
 - use a library?
 - create a design pattern?



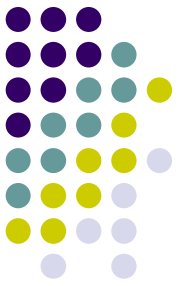
Prototype to Learn Magic Elf (coding)



- Ask yourself
 - What scares you?
- If a magic coding elf,
 - What mini reference code sample would like to see?
 - Give that to-do list to the elf
- Keep creating a list
 - Until you have everything unsure of is on the list
- Now let's go find that elf



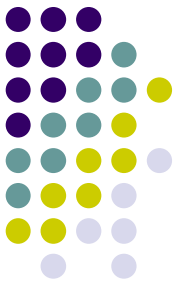
Prototype to Learn Code structure



- How do I code?
 - Quick and dirty
- No need to make code
 - Beautiful
 - Just working
- Layout and structure
 - One solution
 - Many projects
 - Many solutions
 - Individual projects

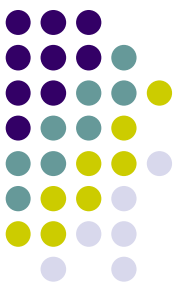


Prototype to Learn Change Roles



- Now put on your elf hat.
 - **YOU** are that elf
- Start doing mini projects for each task on list
 - Do the scariest first
- What happens if it's still too hard.
 - Coarse to fine – my friend
 - Break into still smaller parts





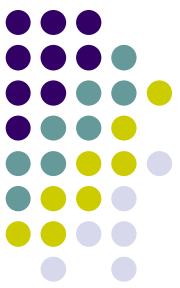
Identify Use Cases

- Designs are cool,
 - but if you can't use them...
- Determine the most popular use patterns
 - Create pseudo code
 - Write sample code to invoke methods
 - Create a use case that will be repeated in practice
- Streamline your design to optimized the user's experience
 - If there is a pattern the users are doing repeatedly
 - Make sure that is easy and intuitive to use
 - Always affects design



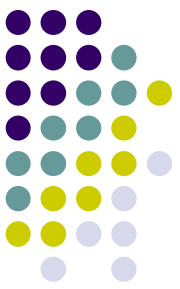
Identify Use Cases

Speculation



- Don't speculate on future Use Cases
 - Only code for the patterns actually used
 - Specific interactions 80% expected
- Future, “What if” Use Cases
 - Can send you on a goose chase
 - That's why there is Refactoring
- Target the 80%
 - Make sure 80% common cases are easy to use
 - Do not scare users away



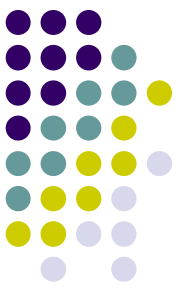


Tracer Bullets to Target

- Tracer Bullets help
 - Fast immediate feedback
 - Find unknown issues
 - Identify holes in Design
 - Create Framework
- Attack areas that scare you
 - Force you to confront the **UNKNOWN**

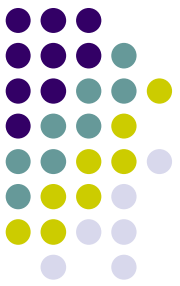


Tracer Bullets to Target Advantages

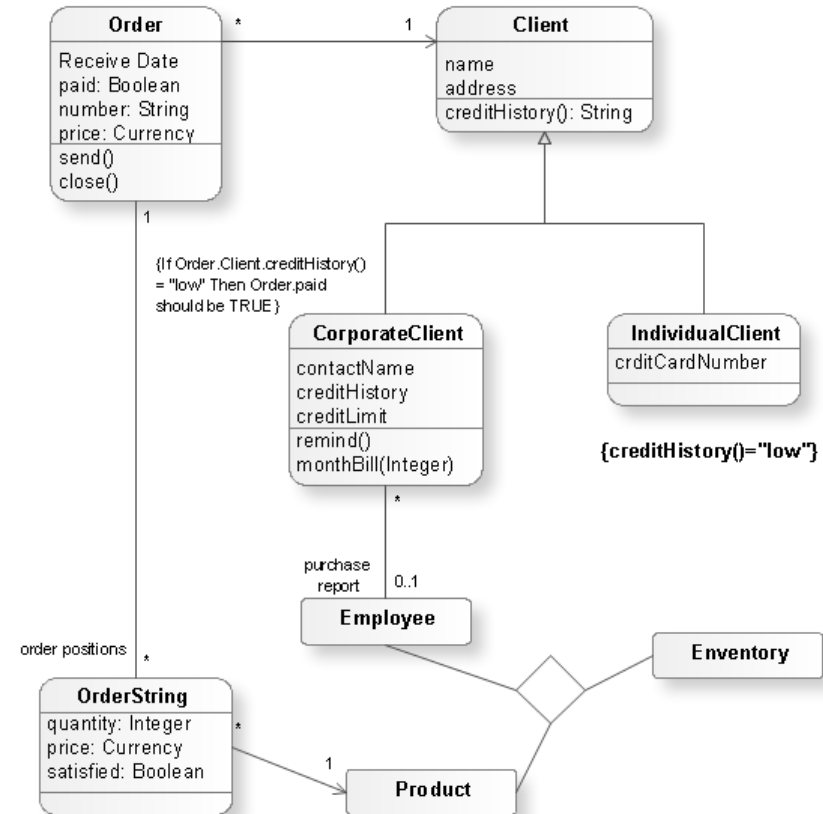


- Users see something working early
 - Visible progress toward a working system
- Developers build a structure
 - Worked out all the end-to-end interactions of your application
- You have an integration platform.
 - Environment to which you can add new pieces of code
- You have something to demonstrate
 - Have something to show
- You have a better feel for progress.
 - Easier to measure performance and to demonstrate progress

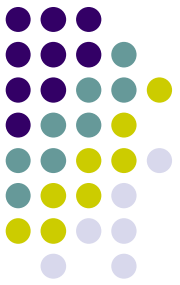
Tracer Bullets to Target Design Phase



- Tracer on Paper
 - UML
 - Interactions
- Coarse to Fine
 - High level
 - Break down into smaller parts
 - Continue until easy component

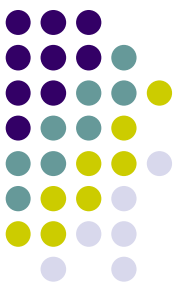


Tracer Bullets to Target Code



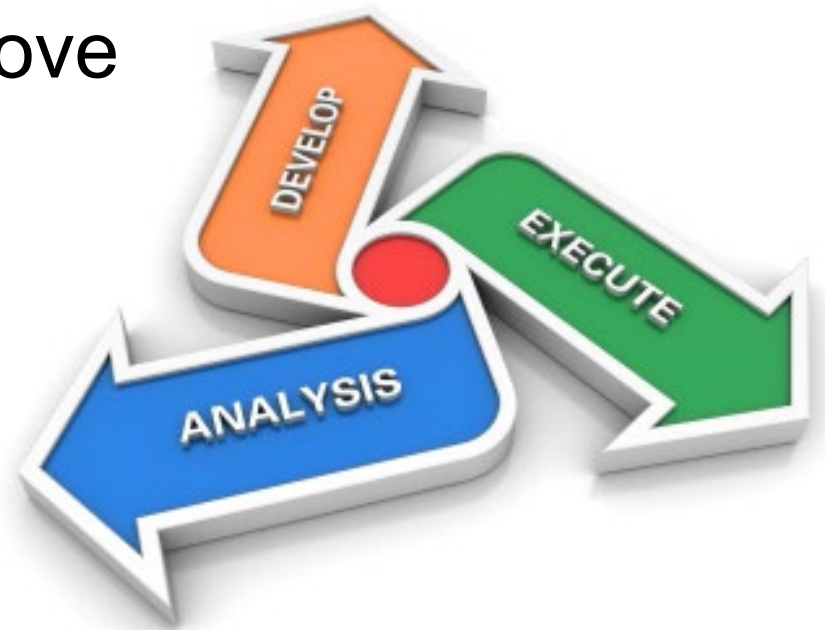
- Quickly write the system
 - Stub out classes
 - Add print statements
 - Pass dummy data back and forth
 - Discover Missing components/systems
 - Try to connect end to end
 - Creates a Framework
 - Iterate on framework





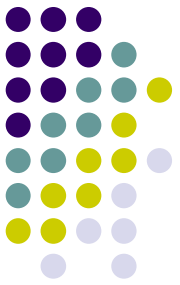
Iterate

- Incrementally improve your system
 - Once you have a framework working
- Identify Section to improve
 - Design
 - Implement
 - Test
 - Refactor
- Repeat



Iterate

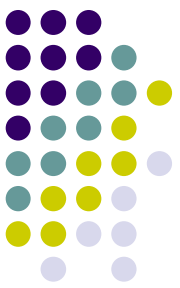
Baby Steps



- Take Baby Steps
 - What are baby steps?
 - <http://www.youtube.com/watch?v=ncFCdCjBqcE>

Iterate

Perforce Your Friend

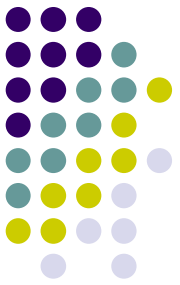


- Improve one section
 - Submit to Perforce
 - May discrete check-ins
- If broke
 - Go back through previous version to find problem
 - Small check-ins
 - Easier to Tree that Bug
 - like a coon dog

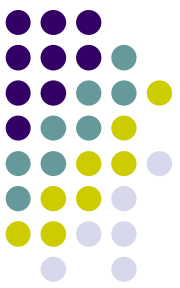


Iterate

Test Driven Development



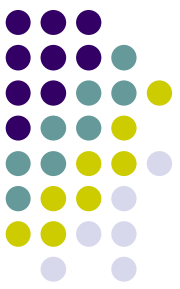
- Write a Unit test or Code sample
 - With the use case
 - Test fails
- Add code to make tests pass
 - Once passed
 - Submit to perform
- Repeat



Never Finished

- Your system is NEVER finished
 - Still improve even if it works
- Areas to improve
 - Refactoring to improve design
 - Simply and reduce complexity
 - Orthogonality
 - Change resource usage
 - Improve efficiency
 - Improve maintainability





Methodology Comparison

Keenan's Process

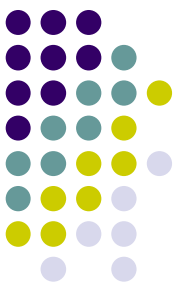
- Outline Features
- Prototype to Learn
- Indentify Use Cases
- Tracer Bullets
- Iterate

SE Process

- Gather Requirements
- Design
- Implement
- Test

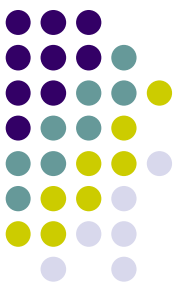
Methodology Comparison

Differences



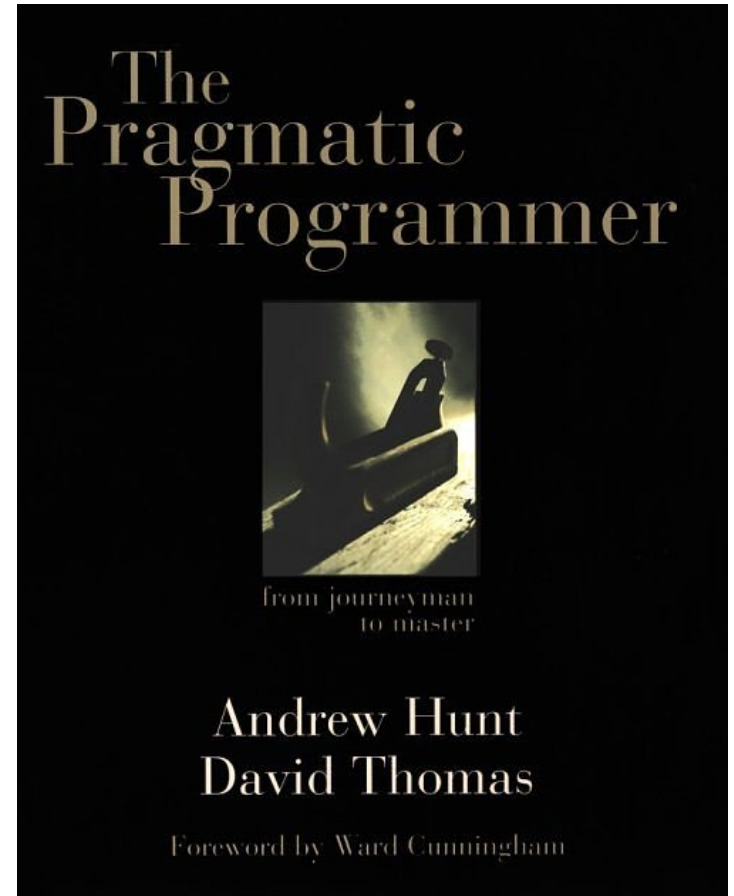
- What did I change?
- Added a process
 - To understand the **Unknown**
 - For discovery
 - To react to what's working or not
 - Stayed pragmatic



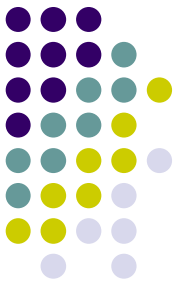


Great Book

- Pragmatic Programmer
 - Great book
 - Must Read
- Captures 90% of my philosophy in one place
- DePaul library has ebook by Safari



Thank You!



- Questions?

